PEN:- 240840116021

Practical-2

Aim:- Perform data inspection, cleaning, handling missing values, duplicate removal, and data transformation using Pandas

Step 1: Import pandas library

```
## Import pandas so we can work with tables and data
import pandas as pd
```

Step 2 : Create DataSet

```
# Create a dictionary containing student information
data = {
    "ID":[101,102,103,104,104,105,106,107],      # Student ID
    "Name": ["Rahul", "Priya", "Amit", None, "Riya", "Ajay", "Naresh", "Jainil"],      # Student names
    "Age": [20, 21, None, 20, 20, 22, 23, 28],  # Student ages
    "City": ["Surat", "Navsari", "Surat", "Bharuch", "Bharuch", "Navsari", "Surat", "Ahmedabad"],     # Student cities
    "Marks": [85, None, 78, 88, 88, 81, 90, 93]  # Student marks

}
```

Step 3 : Create DataFrame

```
# Convert the dictionary 'data' into a Pandas DataFrame
df=pd.DataFrame(data)

# Display the DataFrame
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.0
1	102	Priya	21.0	Navsari	NaN
2	103	Amit	NaN	Surat	78.0
3	104	None	20.0	Bharuch	88.0
4	104	Riya	20.0	Bharuch	88.0
5	105	Ajay	22.0	Navsari	81.0
6	106	Naresh	23.0	Surat	90.0
7	107	Jainil	28.0	Ahmedabad	93.0

Next steps: Generate code with df | New interactive sheet

step 4 : Display first 5 record

```
# Display the first 5 rows of the dataset
df.head()
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.0
1	102	Priya	21.0	Navsari	NaN
2	103	Amit	NaN	Surat	78.0
3	104	None	20.0	Bharuch	88.0
4	104	Riya	20.0	Bharuch	88.0

Next steps: Generate code with df | New interactive sheet

step 5 : Display last 5 record

```
# Display the last 5 rows of the dataset
df.tail()
```

	ID	Name	Age	City	Marks
3	104	None	20.0	Bharuch	88.0
4	104	Riya	20.0	Bharuch	88.0
5	105	Ajay	22.0	Navsari	81.0
6	106	Naresh	23.0	Surat	90.0
7	107	Jainil	28.0	Ahmedabad	93.0

step 6 : Display Information

```
# Display information about the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   ID      8 non-null      int64  
 1   Name    7 non-null      object 
 2   Age     7 non-null      float64
 3   City    8 non-null      object 
 4   Marks   7 non-null      float64
dtypes: float64(2), int64(1), object(2)
memory usage: 452.0+ bytes
```

step 7 : Display Statistical Summary

```
# Display statistical summary of the numerical columns
df.describe()
```

	ID	Age	Marks
count	8.00	7.000000	7.000000
mean	104.00	22.000000	86.142857
std	2.00	2.886751	5.209881
min	101.00	20.000000	78.000000
25%	102.75	20.000000	83.000000
50%	104.00	21.000000	88.000000
75%	105.25	22.500000	89.000000
max	107.00	28.000000	93.000000

Step 8 : Display Dimension of Dataset

```
# Display the number of rows and columns in the dataset
df.shape
```

```
(8, 5)
```

Step 9 : Display Name of column

```
# Display the names of all columns in the dataset
df.columns
```

```
Index(['ID', 'Name', 'Age', 'City', 'Marks'], dtype='object')
```

Step 10 : Display DataType of each column

```
# Display the data type of each column
df.dtypes
```

	0
ID	int64
Name	object
Age	float64
City	object
Marks	float64

dtype: object

Step 11 : Check missing values

```
# Count the number of missing (null) values in each column
df.isnull().sum()
```

	0
ID	0
Name	1
Age	1
City	0
Marks	1

dtype: int64

Step 12 : Fill missing values in Marks column

```
# Fill the missing values in the Marks column with the mean of Marks
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

# Display the updated DataFrame
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102	Priya	21.0	Navsari	86.142857
2	103	Amit	NaN	Surat	78.000000
3	104	None	20.0	Bharuch	88.000000
4	104	Riya	20.0	Bharuch	88.000000
5	105	Ajay	22.0	Navsari	81.000000
6	106	Naresh	23.0	Surat	90.000000
7	107	Jainil	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 13 : Fill missing values in Age column

```
# Fill missing values in the "Age" column with the average (mean) age
df["Age"] = df["Age"].fillna(df["Age"].mean())

# Display the updated DataFrame
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102	Priya	21.0	Navsari	86.142857
2	103	Amit	22.0	Surat	78.000000
3	104	None	20.0	Bharuch	88.000000
4	104	Riya	20.0	Bharuch	88.000000
5	105	Ajay	22.0	Navsari	81.000000
6	106	Naresh	23.0	Surat	90.000000
7	107	Jainil	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 14 : Fill missing values in Name column

```
# Fill missing values in the "Name" column with "unknown"
df["Name"] = df["Name"].fillna("unknown")

# Display the updated DataFrame
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102	Priya	21.0	Navsari	86.142857
2	103	Amit	22.0	Surat	78.000000
3	104	unknown	20.0	Bharuch	88.000000
4	104	Riya	20.0	Bharuch	88.000000
5	105	Ajay	22.0	Navsari	81.000000
6	106	Naresh	23.0	Surat	90.000000
7	107	Jainil	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 15:Detect Duplicate records

```
# Check for duplicate rows in the DataFrame
# Returns True for rows that are duplicates
df.duplicated()
```

	0
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False

dtype: bool

Step 16 : Count total no of Duplicate record

```
# Count the total number of duplicate rows in the DataFrame
df.duplicated().sum()
```

```
np.int64(0)
```

Step 17 : Remove Duplicate column

```
# Remove all duplicate rows from the DataFrame
df = df.drop_duplicates()

# Display the updated DataFrame
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102	Priya	21.0	Navsari	86.142857
2	103	Amit	22.0	Surat	78.000000
3	104	unknown	20.0	Bharuch	88.000000
4	104	Riya	20.0	Bharuch	88.000000
5	105	Ajay	22.0	Navsari	81.000000
6	106	Naresh	23.0	Surat	90.000000
7	107	Jainil	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 18 :Rename column Marks to Score

```
# Rename the "Marks" column to "Score"
df.rename(columns={"Marks": "Score"}, inplace=True)

# Display the updated DataFrame
df
```

	ID	Name	Age	City	Score
0	101	Rahul	20.0	Surat	85.000000
1	102	Priya	21.0	Navsari	86.142857
2	103	Amit	22.0	Surat	78.000000
3	104	unknown	20.0	Bharuch	88.000000
4	104	Riya	20.0	Bharuch	88.000000
5	105	Ajay	22.0	Navsari	81.000000
6	106	Naresh	23.0	Surat	90.000000
7	107	Jainil	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 19 : Convert Age colum Datatype from float to int

```
# Convert the "Age" column from its current data type to integer
df["Age"] = df["Age"].astype(int)

# Display the data types of all columns
df.dtypes
```

	0
ID	int64
Name	object
Age	int64
City	object
Score	float64

dtype: object

Step 20 : Create new column Final score

```
# Add 5 marks to the "Score" column and store the result in a new column
df["Final score"] = df["Score"] + 5
```

```
# Display the updated DataFrame
df
```

	ID	Name	Age	City	Score	Final score
0	101	Rahul	20	Surat	85.000000	90.000000
1	102	Priya	21	Navsari	86.142857	91.142857
2	103	Amit	22	Surat	78.000000	83.000000
3	104	unknown	20	Bharuch	88.000000	93.000000
4	104	Riya	20	Bharuch	88.000000	93.000000
5	105	Ajay	22	Navsari	81.000000	86.000000
6	106	Naresh	23	Surat	90.000000	95.000000
7	107	Jainil	28	Ahmedabad	93.000000	98.000000

Next steps: Generate code with df | New interactive sheet

Step 21 : Convert all student name to uppercase

```
# Convert all names in the "Name" column to uppercase
df["Name"] = df["Name"].str.upper()
df
```

	ID	Name	Age	City	Score	Final score
0	101	RAHUL	20	Surat	85.000000	90.000000
1	102	PRIYA	21	Navsari	86.142857	91.142857
2	103	AMIT	22	Surat	78.000000	83.000000
3	104	UNKNOWN	20	Bharuch	88.000000	93.000000
4	104	RIYA	20	Bharuch	88.000000	93.000000
5	105	AJAY	22	Navsari	81.000000	86.000000
6	106	NARESH	23	Surat	90.000000	95.000000
7	107	JAINIL	28	Ahmedabad	93.000000	98.000000

Next steps: Generate code with df | New interactive sheet

Step 22 : Convert city name to lowercase

```
# Convert all city names in the "City" column to lowercase
df["City"] = df["City"].str.lower()
df
```

	ID	Name	Age	City	Score	Final score
0	101	RAHUL	20	surat	85.000000	90.000000
1	102	PRIYA	21	navsari	86.142857	91.142857
2	103	AMIT	22	surat	78.000000	83.000000
3	104	UNKNOWN	20	bharuch	88.000000	93.000000
4	104	RIYA	20	bharuch	88.000000	93.000000
5	105	AJAY	22	navsari	81.000000	86.000000
6	106	NARESH	23	surat	90.000000	95.000000
7	107	JAINIL	28	ahmedabad	93.000000	98.000000

Next steps: Generate code with df | New interactive sheet

Step 23 : Remove leading and trailing whitespace from City column

```
# Remove extra spaces from the beginning and end of city names
df["City"] = df["City"].str.strip()
df
```

	ID	Name	Age	City	Score	Final score
0	101	RAHUL	20	surat	85.000000	90.000000
1	102	PRIYA	21	navsari	86.142857	91.142857
2	103	AMIT	22	surat	78.000000	83.000000
3	104	UNKNOWN	20	bharuch	88.000000	93.000000
4	104	RIYA	20	bharuch	88.000000	93.000000
5	105	AJAY	22	navsari	81.000000	86.000000
6	106	NARESH	23	surat	90.000000	95.000000
7	107	JAINIL	28	ahmedabad	93.000000	98.000000

Next steps: Generate code with df | New interactive sheet

Step 24 : Sort data based on Final score in Descending order

```
# Sort the DataFrame by "Final score" in descending order
# Highest score will appear first
df_sorted = df.sort_values(by="Final score", ascending=False)

# Display the sorted DataFrame
df_sorted
```

	ID	Name	Age	City	Score	Final score
7	107	JAINIL	28	ahmedabad	93.000000	98.000000
6	106	NARESH	23	surat	90.000000	95.000000
3	104	UNKNOWN	20	bharuch	88.000000	93.000000
4	104	RIYA	20	bharuch	88.000000	93.000000
1	102	PRIYA	21	navsari	86.142857	91.142857
0	101	RAHUL	20	surat	85.000000	90.000000
5	105	AJAY	22	navsari	81.000000	86.000000
2	103	AMIT	22	surat	78.000000	83.000000

Next steps: Generate code with df_sorted | New interactive sheet

Step 25 : Save cleaned DataFrame as new CSV file.

```
# Save the cleaned DataFrame as a CSV file
# index=False prevents pandas from adding the row numbers as a separate column
df.to_csv("cleaned-students.csv", index=False)

# Display a success message
print("Dataset Created Successfully")
```